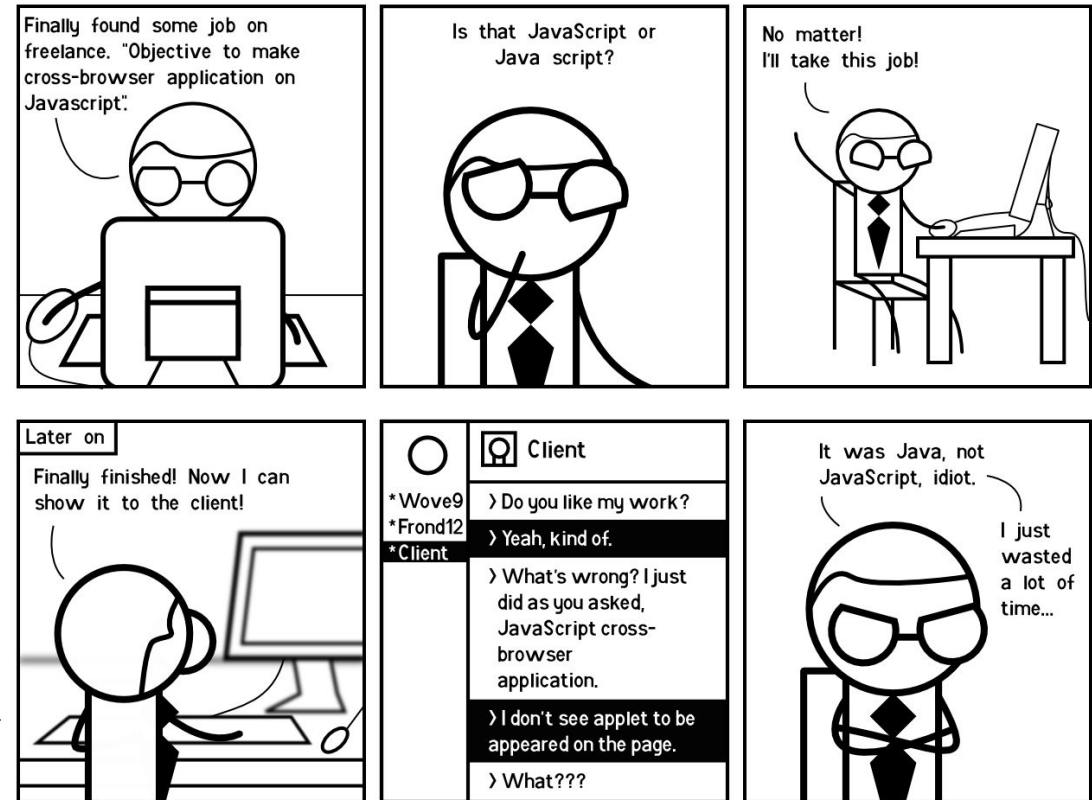


Arrays & Event Driven

LECTURE 7(A): JAVASCRIPT



Arrays

```
var name = []; // empty array
var name = [value, value, ..., value]; // pre-filled
name[index] = value; // store element PHP
```

```
var ducks = ["Huey", "Dewey", "Louie"];

var stooges = []; // stooges.length is 0
stooges[0] = "Larry"; // stooges.length is 1
stooges[1] = "Moe"; // stooges.length is 2
stooges[4] = "Curly"; // stooges.length is 5
stooges[4] = "Shemp"; // stooges.length is 5 PHP
```

- two ways to initialize an array
- **length** property (grows as needed when elements are added)

Array methods

```
var a = ["Stef", "Jason"]; // Stef, Jason
a.push("Brian");           // Stef, Jason, Brian
a.unshift("Kelly");        // Kelly, Stef, Jason, Brian
a.pop();                   // Kelly, Stef, Jason
a.shift();                 // Stef, Jason
a.sort();                  // Jason, Stef
```

JS

- array serves as many data structures: list, queue, stack, ...
- methods: concat, join, pop, push, reverse, shift, slice, sort, splice, toString, unshift
 - push and pop add / remove from back
 - unshift and shift add / remove from front
 - shift and pop return the element that is removed

Splitting strings: split and join

```
var s = "the quick brown fox";  
var a = s.split(" ");           // ["the", "quick", "brown", "fox"]  
a.reverse();                    // ["fox", "brown", "quick", "the"]  
s = a.join("!");                // "fox!brown!quick!the"      JS
```

- split breaks apart a string into an array using a delimiter
 - can also be used with regular expressions surrounded by /:

```
var a = s.split(/[ \t]+/);
```

- join merges an array into a single string, placing a delimiter between them

Defining functions

```
function name() {  
    statement ;  
    statement ;  
    ...  
    statement ;  
}
```

JS

```
function myFunction() {  
    alert("Hello!");  
    alert("How are you?");  
}
```

JS

- the above could be the contents of `example.js` linked to our HTML page
- statements placed into functions can be evaluated in response to user events

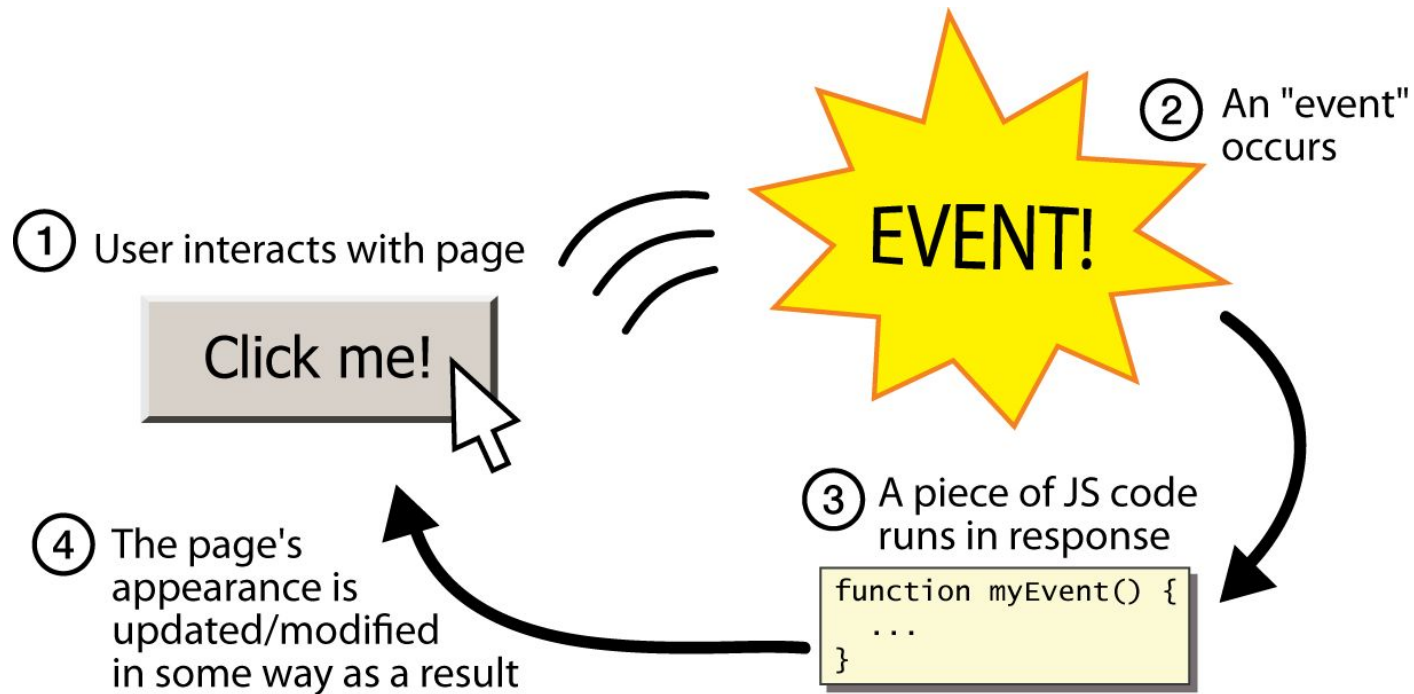
Special values: null and undefined

```
var ned = null;  
var benson = 9;  
var caroline;  
  
// at this point in the code,  
//   ned is null  
//   benson's 9  
//   caroline is undefined
```

JS

- `undefined` : has not been declared, does not exist
- `null` : exists, but was specifically assigned an empty or `null` value
- Why does JavaScript have both of these?

Event-driven programming



- JS programs have no `main`; they respond to user actions called **events**
- **event-driven programming**: writing programs driven by user events

Event handlers

<code><element attributes onclick="function();">...</code>	HTML
<code><div onclick="myFunction();">Click me!</div></code>	HTML
Click me!	HTML

- JavaScript functions can be set as **event handlers**
 - when you interact with the element, the function will execute
- onclick is just one of many event HTML attributes we'll use

Buttons: <button>

the canonical clickable UI control (inline)

```
<button onclick="myFunction();" >Click me!</button>
```

HTML

output

- button's text appears inside tag; can also contain images
- To make a responsive button or other UI control:
 1. choose the control (e.g. button) and event (e.g. mouse click) of interest
 2. write a JavaScript function to run when the event occurs
 3. attach the function to the event on the control

Accessing an element: document.getElementById

```
var name = document.getElementById("id");
```

JS

```
  
<button onclick="changeImage();">Click me!</button>
```

HTML

```
function changeImage() {  
    var octopusImage = document.getElementById("icon01");  
    octopusImage.src = "images/kitty.gif";  
}
```

JS



Click me!

output

- `document.getElementById` returns the DOM object for an element with a given `id`